

MAC5739 - Inteligência Artificial (2026)
Segundo Exercício-Programa
Busca Adversarial para o Pac-Man

IMPORTANTE: EP obrigatório para alunos da Pós-Graduação e optativo para alunos da Graduação

12 de maio de 2026

1 Introdução

Neste exercício-programa estudaremos algoritmos de busca adversarial para o jogo [Pac-Man](#), sendo uma continuação do EP1. Utilizaremos parte do material/código livremente disponível do curso UC Berkeley CS188 ¹.

Esse segundo EP é obrigatório para os alunos de Pós-graduação e optativo para os alunos da Graduação. Caso o aluno da graduação opte por não entregá-lo, não haverá prejuízo para sua nota final.

O objetivo deste exercício-programa será implementar algoritmos de h-minimax, h-minimax com alpha-beta pruning e expectimax.

Neste problema, iremos levar em consideração explicitamente os fantasmas no jogo e a comida a ser coletada (figura 1. Caso não conheça o jogo e precise de mais detalhes do seu funcionamento, veja a descrição do EP1.

¹<https://inst.eecs.berkeley.edu/~cs188/archive/fa25/>

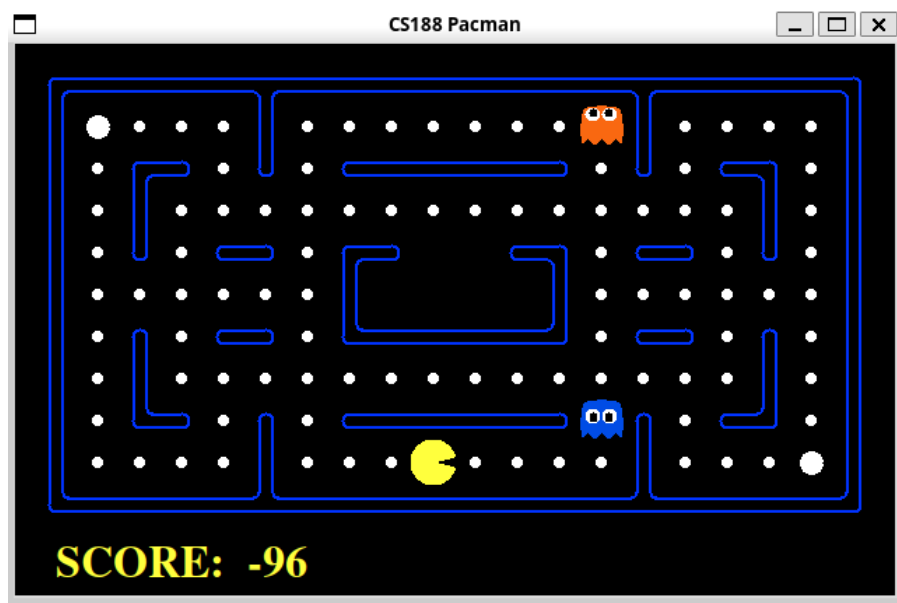


Figura 1: Exemplo de jogo do Pac-Man com fantasmas.

1.1 Instalação

Para a realização deste EP será necessário ter instalado em sua máquina a versão 3.9 do Python ². Será também necessário o pacote *tkinter*. Verifique se ele já está instalado usando o comando:

```
$ python -m tkinter
```

Caso não esteja, instale-o no seu sistema separadamente ou modifique sua instalação do Python para incluí-lo.

Não é necessário usar os seus arquivos do EP1. Em vez disso, faça o download do código novo no **arquivo ep2.zip** disponível na página da disciplina no e-Disciplinas. Descompacte o arquivo ep2.zip e rode na raiz do diretório o seguinte comando para testar a instalação:

```
$ cd multiagent/  
$ python pacman.py
```

Após executar esta linha de comando, uma nova janela deve ter sido aberta em seu desktop com o simulador sendo executado e o agente do Pac-Man parado (pois chamamos a linha de comando sem nenhum algoritmo especificado).

2 Implementação

Nesse exercício-programa, irá implementar o método `self.getAction(gameState)` em diferentes classes de agente. Esse método deverá retornar a próxima ação que o Pac-Man irá executar no estado

²<https://www.python.org/downloads/>

atual (`gameState`). O único arquivo a ser modificado será o **multiagent/multiAgents.py**. Na implementação, você deverá usar alguns métodos do estado do jogo. São esses:

- `gameState.getLegalActions(agentIndex)`: Devolve a lista de ações válidas para o dado agente, que é identificado por um índice não-negativo. O Pac-Man (jogador) é o `agentIndex=0`, e os demais índices são os fantasmas.
- `gameState.getNumAgents()`: Devolve o número total de agentes no jogo (incluindo o próprio Pac-Man).
- `gameState.generateSuccessor(agentIndex, action)`: Devolve o estado de jogo sucessor ao aplicar a dada ação.
- `gameState.isWin()`: Devolve se o estado é final e o jogador venceu (coletou toda a comida).
- `gameState.isLose()`: Devolve se o estado é final e o jogador perdeu (foi pego por um fantasma).
- `gameState.getScore()`: Devolve o score atual do jogador.
- `self.evaluationFunction(gameState)`: Devolve um número indicando uma estimativa da utilidade do dado estado.

Veja que os métodos acima são semelhantes a algumas das funções usadas nos slides da Aula 10 disponível no e-disciplinas:

- `gameState.isWin()` e `gameState.isLose()` equivalem ao **TESTE-DE-TÉRMINO(estado)** nos slides;
- `gameState.generateSucessor(agentIndex, action)` e `gameState.getLegalActions(agentIndex)` são semelhantes às funções **RESULTADO(estado,a)** e **AÇÕES(estado)**, respectivamente, com a diferença de que o agente deve ser especificado explicitamente.
- `gameState.getScore()` equivale à função **UTILIDADE(estado)**;
- `self.evaluationFunction(gameState)` equivale à função **AVAL(estado)**.

Os turnos dos agentes são em ordem de `agentIndex`, portanto o Pac-Man age primeiro, depois os fantasmas. As ações de um agente são apenas cinco: uma para cada direção de movimento (`Directions.NORTH`, `Directions.SOUTH`, `Directions.EAST` e `Directions.WEST`) e uma de ficar parado (`Directions.STOP`). Se, ao mover para uma célula, ele encostar em uma comida, essa comida é coletada automaticamente. Uma ação se torna impossível quando há uma parede na célula vizinha na direção do movimento.

Observação: Diferente do EP1, os estados são sempre do tipo `GameState`. Você não precisará simplificar o estado para os algoritmos de busca.

3 Parte prática

Você deverá implementar os seguintes algoritmos de busca: H-Minimax, H-Minimax com Alpha-Beta pruning e Expectimax com profundidade limitada. Para isso, você deverá implementar algumas funções no arquivo **multiAgents.py** (sinalizadas pelo comentário ***** ADD YOUR CODE HERE *****).

Note que a árvore do jogo Pac-Man pode ser muito grande e portanto não poderemos implementar os algoritmos de busca adversarial até os nós terminais. Portanto, precisa fazer uma poda na profundidade da árvore de jogo e tratar nós não-terminais como se fossem folhas. Para isso, usaremos duas funções de avaliação (dadas a seguir).

Os três agentes são subclasses de `MultiAgentSearchAgent`. Essa classe contém dois campos customizáveis:

- `self.depth`: Limita a árvore de busca. Diferente dos slides, a profundidade diz respeito apenas às *ações do Pac-Man*. Portanto, nos nós de fantasmas, **não devemos incrementar a profundidade**.
- `self.evaluationFunction`: Por padrão, é a função `scoreEvaluationFunction`, que retorna o valor de `gameState.getScore()`. No entanto, você verá que esse valor pode não ser suficiente para guiar o comportamento do agente. Também disponibilizamos uma segunda função de avaliação mais informada, chamada de `betterEvaluationFunction`, que está disponível para ser usada no lugar da função `scoreEvaluationFunction`.

Os cenários (layouts) que você deverá usar para teste são quatro: `minimaxClassic`, `mediumClassic`, `openClassic` e `trappedClassic`. Para testar em cada um desses layouts, use a opção “-l nomeDoMapa”. Para rodar várias partidas de uma só vez só, use a opção “-n numeroDePartidas”.

Por padrão, os fantasmas se movem aleatoriamente, mas você pode usar a opção -g `DirectionalGhost` para fazer o fantasma perseguir o Pac-Man. Para fixar o comportamento dos fantasmas aleatórios, você pode usar a opção -f para fixar a seed aleatória. Note que o comportamento dos fantasmas é definido separadamente do agente do Pac-Man, então o seu algoritmo não será usado para os fantasmas agirem, apenas para o Pac-Man.

Código 1 - Algoritmo H-Minimax

Implemente o algoritmo H-Minimax no método `getAction` da classe `MinimaxAgent`. Defina funções auxiliares se precisar. Para testar seu algoritmo rode o comando:

```
$ python pacman.py -p MinimaxAgent -l minimaxClassic -a depth=4
```

Teste também com diferentes níveis de profundidade. No layout `minimaxClassic`, o Pac-Man deve vencer na maioria dos casos, mas não necessariamente em todos. Em layouts maiores como `openClassic` e `mediumClassic`, você perceberá que ele consegue evitar morrer, mas não sabe o que fazer para ganhar e pode ficar parado até quando tem comida do lado dele. Isso pode ser melhorado usando a outra função de avaliação. Para isso, teste com o comando:

```
$ python pacman.py -p MinimaxAgent -l minimaxClassic -a depth=4,evalFn=better
```

Observação: Não esqueça de remover o código `util.raiseNotDefined()` ao final de cada função.

Teste também seu algoritmo no layout `trappedClassic` e tente entender o comportamento do Pac-Man nesse caso:

```
$ python pacman.py -p MinimaxAgent -l trappedClassic -a depth=3
```

Dica: Se o Pac-Man se mover muito devagar, tente usar a opção “`--frameTime 0`” na linha de comando. Se o jogo demorar muito para terminar, use a opção “`-q`” para pular a simulação gráfica.

Código 2 - H-Minimax com Alpha-Beta Pruning

Implemente o algoritmo H-minimax com alpha-beta pruning no método `getAction` da classe `AlphaBetaAgent`. Para testar seu algoritmo, rode o comando:

```
$ python pacman.py -p AlphaBetaAgent -l minimaxClassic -a depth=4
```

Observação: Diferente dos slides, você **não deve fazer a poda no caso de igualdade**, pois isso pode comprometer a avaliação do autograder.

Código 3 - Expectimax com profundidade limitada

Implemente o algoritmo expectimax com profundidade máxima (`self.depth`) no método `getAction` da classe `ExpectimaxAgent`. Para testar seu algoritmo, rode o comando:

```
$ python pacman.py -p ExpectimaxAgent -l minimaxClassic -a depth=3
```

Compare com o comportamento do `AlphaBetaAgent` do `trappedClassic`:

```
$ python pacman.py -p ExpectimaxAgent -l trappedClassic -a depth=3 -q -n 10
$ python pacman.py -p AlphaBetaAgent -l trappedClassic -a depth=3 -q -n 10
```

4 Relatório

Após o desenvolvimento da parte prática, você deverá testar seus algoritmos e redigir um relatório, claro e sucinto (de 1 a 2 páginas, no máximo), fazendo uma análise comparativa de desempenho dos algoritmos implementados *nos quatro layouts de teste usando dois tipos de fantasmas*: aleatório padrão e o adversarial (–g `DirectionalGhost`). Assim, você deverá:

- compilar em tabelas a *pontuação final* obtida pelos três algoritmos implementados. Teste duas variações de cada algoritmo: com função de avaliação padrão e com a `evalFn=better`. Teste com as profundidades máximas 1 e 3. Para comparar os algoritmos em termos dos scores finais, rode cada caso de teste *três vezes* e pegue a média do score.
- discutir as vantagens e desvantagens de cada algoritmo em cada caso de teste, bem como explicar comportamento do agente neles.

5 Entrega

Você deve entregar **dois arquivos** separados:

- O arquivo `multiAgents.py` modificado com as suas implementações;

- (2) Um arquivo chamado `rel-EP1-NUSP-NOME-COMPLETO.pdf` em formato PDF contendo o relatório com as comparações e discussão dos resultados (máximo de 2 páginas).

Não esqueça de identificar cada arquivo com seu nome e número USP! Faça isso tanto no código (coloque um cabeçalho em forma de comentário) quanto no relatório (coloque em destaque no título).

6 Critério de avaliação

O critério de avaliação atribuirá pesos iguais à parte prática e ao relatório. A avaliação da parte prática dependerá parcialmente dos resultados dos testes automatizados do `autograder.py`. Dessa forma você terá como avaliar por si só parte da nota que receberá para a parte prática. Para rodar os testes automatizados pelo `autograder`, execute o seguinte comando:

```
$ python autograder.py
```

A saída do `autograder` é a pontuação obtida em cada questão incluindo detalhes dos testes que deram errado (caso não tenha passado).

Com relação ao relatório, avaliaremos principalmente sua forma de interpretar comparativamente o desempenho de cada algoritmo em um mesmo cenário. Não é necessário detalhar a estratégia de cada um ou qual estrutura de dados você utilizou, mas deve ficar claro que você compreendeu os resultados obtidos conforme esperado dadas as características teóricas de cada algoritmo.